

Project Report: RAG Pipeline for Job Data Retrieval

1 Introduction

Navigating large repository of job descriptions often suffers from keyword mismatch and lack of structural clarity. To solve this, a semantic RAG engine was constructed. The pipeline transforms messy, raw HTML job descriptions into structured, uniform summaries embedded within a vector database, enabling precise natural language querying coupled with verifiable source citations.

2 Data Pipeline and Preprocessing

The data ingestion pipeline processes raw tabular data containing heterogeneous HTML job postings through two primary transformation stages:

1. **HTML Structural Cleaning:** Using Python's `BeautifulSoup`, raw HTML descriptions are parsed node-by-node. Structural boundaries are preserved by marking probable section headers with `## Header` syntax and list items with bullet markers (`-`). Legal and equal employment opportunity (EEO) boilerplate text is systematically purged using regex pattern matching.
2. **Metadata Ingestion:** A companion script sanitizes pandas dataframe records, removing null/NaN values to ensure compatibility with vector database metadata constraints. It writes out a clean JSONL dataset and seeds the vector database with minimal placeholder records.

3 AI-Powered Summarization Layer

To optimize retrieval quality, raw text is converted into concise, uniform semantic representations rather than embedding long, noisy web blocks.

- **Cloud Integration:** The system interfaces with high-speed cloud LLM APIs to extract structured attributes.
- **Schema Enforcement:** Structured JSON generation is strictly governed by a predefined schema, ensuring every document yields five uniform dimensions: `about_company`, `responsibilities`, `requirements`, `benefits`, and `compensation`.
- **Resiliency & Rate Limiting:** Automatic backoff algorithms and batch pacing handle cloud-tier rate limits, ensuring uninterrupted pipeline execution.

4 Vector Storage and Architecture

The persistence layer utilizes **ChromaDB** configured with cosine similarity metrics (`hnsw:space:cosine`) inside a shared local collection (`job_postings`).

- **Dual-Stage Ingestion Pattern:** The database first stores lightweight metadata placeholders, which are subsequently overwritten by fully summarized rich text chunks during the LLM processing stage.
- **State-Aware Resuming:** The ingestion script checks existing database records by inspecting specific semantic metadata fields (e.g., presence of `responsibilities`). This distinguishes incomplete placeholders from fully processed jobs, allowing the script to safely resume after interruptions without duplicating work.

5 Retrieval and Generation Engine

The RAG retrieval-and-generation module (`rag_engine.py`) acts as the core interface for querying the knowledge base:

1. **Lazy Singleton Connection:** Connects to the persistent vector store efficiently without redundant client instantiations.
2. **Dynamic Metadata Filtering:** User-provided filters (such as job level or location) are dynamically translated into Chroma’s native `$and` query syntax.
3. **Strict Grounding & Citations:** The generation prompt enforces strict hallucination guards, instructing the model to rely exclusively on retrieved context and cite sources via inline bracketed IDs (e.g., [LF0001]).

6 API and Web Interface Implementation

To satisfy deployment and integration requirements, a lightweight production web service was constructed using **FastAPI**. The application exposes a robust POST endpoint at `/api/query`.

Furthermore, a responsive single-page HTML/JavaScript front-end interface is served directly from the root route (`/`). This built-in UI provides an interactive dashboard where users can query the vector database, view real-time LLM generation feedback, and inspect verifiable source citation blocks complete with similarity distance metrics.

7 Conclusion

The implemented RAG framework successfully bridges raw enterprise data and precise semantic search. By combining intelligent HTML parsing, structured schema-constrained LLM summaries, state-aware database synchronization, and tightly grounded generation prompts, the system provides a scalable, accurate, and cost-effective solution for job market analysis.